KernelScript: Exploring a Typed Unified-Source Programming Model for eBPF

Anonymous Authors

Abstract. Realistic eBPF applications span verifier-constrained kernel programs, userspace loaders, shared maps, skeletons, and sometimes kernel-module code. This paper studies KernelScript, a beta domain-specific language (DSL) that explores whether these pieces can be represented in one type-checked source model and lowered to conventional libbpf-compatible artifacts. The scientific question is whether such a unified model can expose cross-boundary eBPF structure to the compiler without abandoning the ordinary Linux BPF toolchain.

At commit 3b19cd2, 43 of 44 repository examples compile from KernelScript source and 41 of 44 build into generated C/eBPF projects. A source-only scan of 4 pinned external eBPF repositories covers 171 files and observes markers for Express Data Path (XDP), traffic control (TC), tracing, maps, ring buffers, struct_ops, and scheduler-extension code in the selected source paths, although it does not translate or build those applications. A manually ported external XDP portfolio from `xdp-tutorial` then compiles from KernelScript: 3 pinned workloads build to eBPF, attach, and pass traffic in both KernelScript and original external C/eBPF form over 5 local trials, and the map-counter workload also increments the same XDP_PASS map key in both variants. The checked-in evaluation then tests selected early rejections, strict verifier loading, isolated XDP attachment, matched XDP and TC traffic, perf-event counters, ring-buffer event delivery, and tcp-congestion struct_ops compatibility. 37 of 41 build-success generated objects verifier-load under a strict pinned-program criterion, and 27 of 27 verifier-clean single-section XDP objects attach and detach in isolated namespaces. For struct_ops, shared libbpf runners distinguish direct object load/attach/detach, loopback TCP byte-transfer behavior, callback reachability under clean and loss-trigger profiles, and a local skeleton-header repair that builds 2/2 affected generated userspace projects. Scheduler-extension experiments add evidence for verifier loading and bounded attachment: both the hand-written C/eBPF object and the generated object verifier-load and pin programs, and an opt-in harness registers both toy schedulers, runs five bounded progress trials, and unregisters them cleanly. The results support an artifact/prototype claim: KernelScript centralizes recurring project structure for this repository corpus and exposes concrete compatibility limits, but it does not yet establish broad runtime equivalence or broad struct_ops portability.

1 Introduction

eBPF gives Linux developers a controlled way to execute application-specific logic in the kernel [11]. Linux developers now use it for packet processing, traffic control, tracing, profiling, security policy, and scheduler experimentation. The programming model remains difficult because the artifact boundary is split: an eBPF program is written under verifier constraints, a userspace loader must open and attach it, maps provide shared state, BPF Type Format (BTF) describes kernel types, and newer interfaces such as kfuncs and struct_ops require close coordination between generated skeleton code and kernel headers.

The usual C/libbpf workflow is powerful but verbose [9]. The developer writes kernel-side C, userspace C, Makefiles, pinning logic, map setup, attachment code, and cleanup paths. Libraries in Rust, Go, or Python improve parts of the workflow [1, 5], but they usually preserve the split between the eBPF program and the controller. Tracing languages such as bpftrace provide a higher-level experience, but intentionally narrow the target domain. The systems question is therefore not whether eBPF needs another surface syntax. The question is whether a language can encode cross-boundary eBPF application structure in a compiler-checkable way that remains compatible with the existing kernel toolchain.

1.1 Why a Unified Source Model?

A single source file is not automatically better than multiple files. The research value comes from making cross-file invariants visible to the compiler. For example, a map declaration is simultaneously a kernel data structure, a userspace file descriptor lookup, a pinning decision, and a type contract between programs. A program reference is simultaneously a source-level function name, a generated BPF program section, a skeleton field, a file descriptor, and possibly a BPF link. In C/libbpf, these relationships are implicit naming conventions and generated-header dependencies. In KernelScript, they can be first-class language values.

This observation motivates the paper's main scientific question: can a typed unified-source model make cross-boundary eBPF application structure explicit to the compiler while preserving the ordinary Linux BPF toolchain path? A tracing-only language such as bpftrace [3] can be concise by excluding XDP, TC, struct_ops, and kfunc workflows. A host-language binding can be ergonomic by focusing on userspace loading. KernelScript attempts a harder point in the design space: preserve multiple eBPF program types and still give the compiler enough structure to generate the loader, maps, and build logic.

KernelScript is a recent DSL that makes this question concrete. It lets developers write eBPF entry points, helper functions, userspace control flow, maps, tail-call style delegation, ring buffers, perf events, kfunc declarations, and kernel-module code in one source language. The compiler lowers this source to eBPF C, userspace C, optional module C, a Makefile, and then the normal libbpf and bpftool path can produce objects, skeletons, and binaries. Because KernelScript is explicitly beta software, its most useful research role today is as a prototype for studying the design space rather than as a production-ready replacement for libbpf.

This paper makes three contributions. First, it formulates KernelScript's central systems idea as a typed unified-source model for multi-artifact eBPF applications, where programs, maps, loaders, pinning choices, and optional kernel-extension code are represented before artifact boundaries are split. Second, it analyzes how the implementation uses that model to check selected cross-boundary invariants, including program-handle use, program signatures, map access types, ring-buffer API use, config mutability, helper-scope restrictions, kernel-context allocation checks, perf-event group bounds, and stack-limit violations before load or attach time. Third, it implements a reproducible artifact evaluation of test coverage, example buildability, generated-code expansion, example marker coverage, external source-corpus feature context, a small manual external port/build/runtime portfolio, static rejection cases, strict verifier loading, isolated XDP attachment, matched XDP and TC traffic runs, perf-event loader and counter workloads, a ring-buffer event-emission workload, direct and callback-instrumented struct_ops checks, and a compiler-source patch that isolates one map-update lowering mechanism. The scientific contribution is not a new BPF verifier or a broad performance-equivalence result. Instead, it is an executable case study of which selected cross-boundary eBPF invariants this prototype exposes and which compatibility limits still leak through the Linux BPF toolchain. The artifact is designed so that a reviewer can rerun the evaluation without changing the KernelScript source tree: all scripts, logs, derived tables, and paper macros are stored outside the cloned repository.

2 Background and Motivation

An eBPF application normally contains at least two components. The first is a restricted kernel program, compiled for the BPF target and accepted only if the kernel verifier can prove memory safety, bounded execution, and legal helper usage. The second is a userspace coordinator that loads objects, finds maps, sets up BPF links, handles signals, and tears resources down. The two components communicate through maps, ring buffers, perf events, or global data.

This split creates three kinds of complexity. **Lifecycle complexity** arises because operations must occur in a valid order: an object must be opened, loaded, attached, read, and detached with type-appropriate APIs. **Representation complexity** appears when the same concept, such as a map or a typed context field, must be represented in source C, generated skeleton headers, userspace lookup code, and BTF-derived type declarations. **Compatibility complexity** appears because eBPF features evolve quickly: struct_ops, kfuncs, dynptrs, TCX (a newer traffic-control attach path), and scheduler extensions depend on kernel, bpftool, and libbpf versions.

The verifier is essential, but it is intentionally late in the pipeline. A developer may write code, compile it, generate skeletons, link a loader, and only then receive a verifier rejection or an attach-time failure. A DSL can help if it moves some of these checks earlier without hiding the underlying eBPF execution model. For example, a language can know that a function marked `@xdp` expects an `xdp_md` context and returns an XDP action, model maps as typed global variables, and distinguish userspace functions from eBPF functions and kfunc definitions.

3 Design Principles

The design principles map directly to the three complexities above. Lifecycle complexity motivates explicit domains and lifecycle typing, representation complexity motivates typed maps, program handles, and generated project structure, and compatibility complexity motivates preserving the kernel toolchain while exposing escape hatches for fast-moving kernel interfaces. The next two sections then show how the language realizes these principles.

Preserve the kernel toolchain. KernelScript does not replace the kernel verifier, clang, bpftool, or libbpf. Instead it generates C and Makefiles that flow through the same toolchain used by C/libbpf projects. This choice gives the prototype immediate access to BTF, skeleton generation, and existing kernel diagnostics. It also means compatibility failures remain visible and measurable.

Make domains explicit. The attribute system separates userspace, eBPF, helper, and kernel-module code. This matters because the same surface syntax cannot mean the same thing in every domain. Userspace can allocate and print freely, eBPF code must obey stack and helper restrictions, and kfunc code targets a module build. A unified source model is useful only if the compiler keeps these domains separate internally.

Move verifier-relevant checks earlier. The compiler performs safety analysis before code generation. In our run, `safety_demo.ks` is rejected because the compiler estimates 608 bytes of stack use, above the 512-byte eBPF limit. The check does not make the verifier redundant, but it can reduce late failures for covered cases that would otherwise require inspecting verifier logs.

Expose low-level escape hatches. eBPF users often need new helpers, kernel functions, or BTF-specific types before high-level libraries expose them. KernelScript supports includes and extern declarations so that a program can refer to generated header declarations. For a research prototype, this escape hatch matters because the Linux BPF API surface changes faster than a DSL can stabilize.

4 KernelScript Overview

KernelScript uses attributes to assign functions to execution domains. A function marked `@xdp`, `@tc`, `@probe`, `@tracepoint`, or `@perf_event` becomes an eBPF entry point. A function marked `@helper` is callable from eBPF code. A normal `main` function is compiled into userspace C. A function marked `@kfunc` can trigger kernel-module generation. The source therefore contains the application structure

that would otherwise be scattered across multiple files.

Listing 1: Minimal shape of a unified-source eBPF application.

```
include "xdp.kh"

@xdp fn pass_all(ctx: *xdp_md) -> xdp_action {
  return XDP_PASS
}

fn main() -> i32 {
  var prog = load(pass_all)
  attach(prog, "lo", 0)
  detach(prog)
  return 0
}
```

This small example already exhibits the three relationships that the next section makes precise: functions are assigned to execution domains by attribute, maps are declared as typed globals shared across domains rather than as separate C sections and userspace lookup code, and the program lifecycle (`load`/`attach`/`detach`) is expressed over typed program handles. The same global-declaration pattern applies to ring buffers and perf events: high-level operations stay explicit in source while the generated code performs the required libbpf calls. All of this structure lives in one source file that the compiler later splits into kernel C, userspace C, skeleton references, and build logic. Section 5 gives the typing rules behind each relationship.

5 Language and Type System

Because the paper's central claim is a *typed* unified-source model, this section specifies the parts of the type system that encode cross-boundary structure. We describe the model informally but precisely; we do not give a full formal semantics or a soundness proof, and we are explicit below about what each rule does and does not guarantee. Every rule in this section corresponds to a concrete diagnostic exercised by the rejection corpus in Section 7 (the file names in parentheses are the cases in `experiments/static_checks/`).

Domains as attributes. A KernelScript program is a flat sequence of declarations. A function attribute assigns each function to an execution domain $d \in \{\textit{user}, \textit{ebpf}, \textit{helper}, \textit{kfunc}\}$: `@xdp`, `@tc`, `@probe`, `@tracepoint`, and `@perf_event` mark eBPF entry points; `@helper` marks kernel-shared eBPF code; `@kfunc` and `@private` mark kernel-module code; an unattributed `fn` is userspace. The type checker tags each function with its domain and rejects calls that cross the domain lattice illegally, for example a userspace function calling an `@helper` (`helper_from_userspace`) or kernel-context allocation requested from a userspace or XDP domain (`gfp_from_userspace`, `gfp_from_xdp`). Each eBPF attribute also fixes a context type and a return type: `@xdp` requires `ctx: *xdp_md` and returns `xdp_action`; `@tc` requires `*__sk_buff` and returns `i32`. A function whose signature disagrees with its attribute is rejected before code generation (`xdp_wrong_context`, `tc_wrong_context`, `xdp_wrong_return`, `probe_wrong_return`).

Maps as typed globals. A map is a typed global declaration `var m : hash<K,V>(N)` or `array<K,V>(N)`, rather than a separate C section plus userspace lookup code. The element types form a contract checked at every access site: an index `m[k]` requires `k : K` and yields a value of type `V`, in both eBPF and userspace domains. Map accesses with the wrong key or value type, or to an undeclared map, are rejected (`map_string_key`, `map_string_value`, `map_undefined`).

Program lifecycle as typestate. The core cross-boundary invariant is program lifecycle. KernelScript distinguishes a `FunctionRef` (a reference to an `@`-attributed function) from a `ProgramHandle` (a loaded program), and types the lifecycle builtins so that only `load` can produce a `ProgramHandle`:

$$\frac{\Gamma \vdash f : \texttt{FunctionRef}}{\Gamma \vdash \texttt{load}(f) : \texttt{ProgramHandle}} \qquad \frac{\Gamma \vdash h : \texttt{ProgramHandle}}{\Gamma \vdash \texttt{detach}(h) : \texttt{unit}}$$

$$\frac{\Gamma \vdash h : \texttt{ProgramHandle} \quad \Gamma \vdash t : \texttt{string},\ \mathit{fl} : \texttt{u32}}{\Gamma \vdash \texttt{attach}(h, t, \mathit{fl}) : \texttt{u32}}$$

Because `attach` and `detach` consume a `ProgramHandle` and only `load` produces one, "attach before load" is a *type* error rather than a runtime failure: passing a bare `FunctionRef` (`attach_without_load`), an integer (`attach_integer_handle`), or a string (`load_string`, `detach_string`) to a lifecycle builtin does not type-check. This is a deliberately lightweight typestate discipline. What it guarantees is that lifecycle builtins are only applied to values of the correct lifecycle type, so the producer/consumer relationship between `FunctionRef` and `ProgramHandle` encodes the intended $\texttt{load} \rightarrow \texttt{attach} \rightarrow \texttt{detach}$ ordering. What it does *not* guarantee is full path-sensitive typestate soundness: the current checker does not statically rule out use-after-detach, double-attach, or leaked handles on every control-flow path. Establishing those properties would require a flow-sensitive analysis we have not implemented, and we therefore treat lifecycle typing as a research direction rather than a closed result.

From invariants to diagnostics. The value of making these relationships typed is that each cross-boundary invariant becomes a localized compiler diagnostic instead of an implicit naming convention or a late verifier or attach-time failure. Section 7 measures the current diagnostic behavior; Section 10 is explicit that the existing corpus tests whether these checks are wired and stable, not yet whether they catch the mistakes developers make in practice.

6 Implementation

The evaluated implementation is an OCaml compiler built with dune. It parses KernelScript source, resolves imports

and includes, builds symbol tables, performs multi-program analysis and type checking, runs safety analysis, generates an intermediate representation, and emits C artifacts. For an input `foo.ks`, the generated project normally contains `foo.ebpf.c`, `foo.c`, a `Makefile`, and, when kfuncs are present, `foo.mod.c`. The Makefile uses bpftool to dump `vmlinux.h`, clang to produce a BPF object, bpftool to generate a skeleton header, and gcc to link the userspace loader with libbpf, libelf, and zlib.

The compiler's architecture aligns with the research hypothesis. It has separate modules for maps, userspace code generation, eBPF C generation, kernel-module generation, context-specific code generation, safety checking, BTF parsing, dynptr bridging, tail-call analysis, and struct_ops registry logic. These modules encode eBPF concepts directly rather than treating the language as a thin preprocessor over C. The design therefore provides places where domain-specific checks can be added: map type compatibility, context field typing, stack estimates, function signature validation, and userspace API generation.

Tail-call orchestration is a concrete example of that design. When a return-position call targets another attributed program with the same program type and compatible signature, the tail-call analysis records the dependency, assigns a prog-array index, and rewrites the call as tail-call metadata in the IR. The eBPF backend then emits a `BPF_MAP_TYPE_PROG_ARRAY` declaration and a `bpf_tail_call()` with a context-appropriate fallback return, while the userspace backend resolves target program file descriptors and populates the prog array at load time. The programmer can therefore write `return drop_handler(ctx)` or a `match` arm such as `TCP: tcp_port_classifier(ctx)` without spelling out the C/libbpf boilerplate directly.

7 Evaluation Methodology

We evaluate KernelScript as a compiler and systems artifact. The goal is to test whether the unified-source model centralizes recurring eBPF project structure, preserves a compile/build path to ordinary libbpf artifacts on our host, and rejects selected pre-load errors. We do not claim packet-rate performance equivalence in this paper, because the current traffic evidence is limited to matched XDP and TC pass/count programs on local veth pairs, and generated-loader runtime evidence is a perf-event lifecycle latency check rather than a dispatch-loop throughput benchmark. Instead, we focus on reproducible evidence that repository examples exercise multiple eBPF domains, selected errors are rejected early, generated artifacts are classified by build and verifier-load outcomes against the local toolchain, matched local workloads cover several program classes, struct_ops checks separate object compatibility, loopback TCP workload behavior, and generated-skeleton compatibility, a skeleton repair checks one version-aware generated-build fix, and one observed runtime gap can be traced to a concrete lowering rule.

The evaluation runs on Linux 6.15.11-061511-generic using clang 18, bpftool 7.7, libbpf 1.3.0, OCaml 4.14, and dune 3.14. The host CPU is a Intel(R) Core(TM) Ultra 9 285K with 24 CPUs, CPU0 governor powersave, turbo enabled, and virtualization reported as none. The microbenchmarks and object workloads do not pin CPUs or change the governor, so their nanosecond and event-rate differences are not statistically interpreted and should be read as local comparisons rather than general performance claims. The evaluated commit is 3b19cd2. The evaluation comprises 23 measurement scripts under `experiments/`, organized around four paper claims. The generated-structure claim uses the example build matrix, matched source-footprint proxy, external source-only scan, and a small manual external port/build/runtime portfolio. The early-checking claim uses unit tests and a targeted static corpus for selected pre-load rejections. The compatibility claim uses strict verifier loading, isolated XDP attach/detach, generated-loader lifecycle checks, object-level perf-event/ring-buffer/struct_ops workloads, and scheduler-extension diagnostics to test local compatibility boundaries. The runtime-behavior claim uses BPF_PROG_TEST_RUN microbenchmarks, matched XDP/TC traffic, longer traffic stress, and lowering ablations to isolate local runtime behavior and one map-update mechanism. The paper reports the compiler-source patch ablation as the canonical mechanism-isolation result because it changes the compiler path rather than editing generated output. The traffic scripts compare generated objects with hand-written C/eBPF baselines built with the same clang/libbpf toolchain and matched for program type, map schema, attachment point, and observable counter or byte-transfer semantics. The baseline policy permits only the idiomatic C/eBPF needed for those matched semantics. The baseline sources are tracked under `experiments/baselines/`. Their construction policy is to match the generated program's observable semantics and avoid C-only shortcuts outside the map operation, attach point, event, or socket behavior under test. The oracles check semantic equivalence at the boundary each workload exposes: exact map counts, perf-counter equality, submitted/received ring-buffer events, byte-transfer completion, load/attach/detach success, or callback flags. The perf-event counter, ring-buffer, and struct_ops scripts use one libbpf runner for both objects and check BPF map counts, submitted/received event counts, load/attach/detach outcomes, TCP socket byte-transfer oracles, or callback-reachability flags. The main mechanism-isolation script copies the KernelScript compiler source, applies a tracked compiler-source patch for array-map increment lowering, rebuilds the patched compiler, and reruns the same count benchmark.

We measure seventeen outcomes across the four claims, and report each one with its full procedure in the correspondingly grouped subsections of Section 8; Table 1 gives the artifact, baseline, oracle, and boundary per claim. For *early checking* we report compiler regression coverage (the repository test suite) and early rejection (static cases whose expected diagnostics are checked by the harness). For *build, load, and compati-*

bility we report example buildability, verifier-load and isolated XDP attach/detach outcomes, direct struct_ops object compatibility, the struct_ops TCP-workload and callback-reachability checks, the struct_ops skeleton repair, the scheduler-extension verifier diagnostic and opt-in attach/progress harness, and generated-loader lifecycle timing. For *generated structure* we report example marker coverage, the generated-structure expansion factor (lines of KernelScript source versus generated C and Makefile lines), a matched tail-call automation case study, the matched source-footprint proxy, the external source-corpus scan, and the external port/build/runtime portfolio. For *runtime behavior* we report the map-update mechanism-isolation ablation and runtime-sanity workloads (tail-call BPF_PROG_TEST_RUN microbenchmarks, matched XDP/TC iperf3 traffic with a longer stress rerun, a perf-event page-fault counter, and a ring-buffer event-emission workload). Two caveats apply throughout and are not repeated at each outcome: example feature labels are derived from lexical markers in the source and are a coarse corpus classifier rather than a feature-completeness audit, and all timing and event-rate numbers are local, unpinned samples on the host described above rather than statistically controlled benchmarks.

The harness treats generated data as first-class evidence. It writes raw stdout and stderr logs for every compiler and make invocation, CSV rows for each example, JSON summaries for scripts, and a small TeX file containing the numeric macros used in this paper. This organization is deliberately close to artifact-evaluation practice: the paper does not rely on a spreadsheet that was manually edited after the run, and the reader can trace each aggregate number to a row in `results/examples_summary.csv`. We also keep temporary build products under `results/build` so that failures can be inspected after the harness exits.

Source lines of code (SLOC) are counted as nonblank, noncomment lines. Generated `vmlinux.h` and generated skeleton headers are excluded from the expansion factor because they are bpftool outputs rather than KernelScript compiler outputs. The expansion therefore counts only source and build logic produced directly by KernelScript: userspace C, eBPF C, optional kernel-module C, and Makefile lines. This definition is conservative for the generated-structure claim because it omits the large skeleton header that a low-level build still needs.

`run_source_footprint.py` adds a separate matched source-footprint proxy for the local matched workloads. It counts nonblank noncomment lines in the maintained KernelScript application source, the hand-written C/eBPF baseline objects, and the C/libbpf runner or loader files when that workload requires one. Generated C, generated Makefiles, generated `vmlinux.h`, skeleton headers, KernelScript library headers, and Python experiment harness code are excluded. This metric is closer to a hand-written baseline than the generated expansion factor, but it is still not a developer-time study.

`run_external_corpus.py` adds an external source-only context check. It fetches pinned commits of `libbpf-bootstrap`, `xdp-tutorial`, and `scx`, scans selected application/example C and header paths, and records file counts, source lines, `SEC()` sections, and feature-family markers. The scan excludes vendored `vmlinux.h` headers, generated files, build outputs, Rust userspace code, and repository support libraries outside the selected application/example paths. The scan is useful for checking whether the local artifact exercises externally visible eBPF feature families, but it is not a porting, build, verifier, or runtime experiment. The script also records a manual classifier spot-check over 8 representative files. The audit compares expected feature markers with the lexical detector output and matches 8 of 8 samples, with 0 false-positive feature labels and 0 false-negative feature labels. The audit is a reconstructability check for the selected marker rules, not a statistical precision/recall estimate.

`run_external_port.py` adds a small external port/build/runtime portfolio. It fetches `xdp-tutorial` commit `4e2bf5658434`, builds manually written KernelScript ports for `basic01-xdp-pass`, `basic02-prog-by-name`, and `basic03-map-counter` through generated Makefiles, and compiles the corresponding original external C/eBPF sources directly with clang. It then attaches each object to a fresh veth in a network namespace and runs iperf3 traffic. The oracle is deliberately semantic and narrow: all workloads must pass traffic, and `basic03-map-counter` must also increment map key XDP_PASS. The experiment does not claim automated translation, upstream-project build coverage, exact packet-count equivalence, or broad external application portability.

8 Results

Result map. The evaluation is organized by four claims. The example build matrix and generated-code expansion test the unified-source structure claim. The unit tests, static rejection corpus, and safety rejection test the early-checking claim. The verifier, attach, perf-event, ring-buffer, and struct_ops checks test local compatibility. The microbenchmarks, compiler patch, traffic runs, and object workloads test runtime behavior and mechanism isolation without claiming broad performance equivalence. The remainder of this section presents the results in four groups that follow these claims: early checking, build/load/attach compatibility, generated structure and source footprint, and runtime behavior.

8.1 Early checking

Test suite. The compiler test suite reports 85 successful suites and 1095 passing tests with no reported failures. The tests cover lexer/parser behavior, type checking, maps, map assignment, userspace code generation, context field typing, safety checking, ring buffers, perf event attach, detach APIs, kfunc attributes, struct_ops, BTF parsing, and other regression cases. This breadth is important because the evaluation of a DSL de-

Table 1: Claim scope, actors, and oracles used in the evaluation.

Claim	Artifact under test	Baseline or actor	Oracle	Boundary
Project generation	Repository examples lowered to generated projects	Compiler and make harness	Build rows, feature markers, generated source lines	Repository corpus only
Source footprint	Matched local workload sources	Hand-written C/eBPF objects and C/libbpf runners	Nonblank noncomment source-line counts with explicit exclusions	Matched source-footprint proxy, not developer time
Tail-call automation	Two matched local XDP dispatch examples	Hand-written C/eBPF objects and C/libbpf loaders	Nonblank noncomment source-line counts plus generated `prog_array`, `bpf_tail_call()`, fallback, and loader-update markers	Two local examples only, not a broad workload sample
External source corpus	Pinned public eBPF example/application repositories	Source-only feature classifier	File/SLOC counts, `SEC()` sections, and feature markers	No translation, build, verifier, attach, or runtime evidence
External port portfolio	3 pinned `xdp-tutorial` XDP sources	Original external C/eBPF sources	Generated-Makefile build for KernelScript ports, direct clang compile for C/eBPF, XDP attach, iperf3 traffic, and map-key increment for the counter workload	Small manual XDP portfolio only, not automated or broad portability
Early checks	Unit tests and targeted invalid programs	Dune and static-check harnesses	Test success and expected diagnostic categories	Selected cases, not soundness
Compatibility	Generated objects and generated perf loader	Local kernel/toolchain	Pinned-program load criterion, attach state, loader attach/read/detach and invocation timing	Local host only
Runtime sanity	Generated XDP, TC, perf-counter, and ring-buffer objects	Matched C/eBPF objects or shared libbpf runner	iperf3 JSON, positive map counts, perf-counter equality, submitted/received/drop counts	Local workloads only
Tcp-congestion struct_ops	Generated and C/eBPF tcp-congestion objects	Shared libbpf runner and socket workload	Load/attach/detach, loopback TCP byte-transfer, clean and loss-injected callback flags	One local tcp-congestion object and selected callback profiles only
Struct_ops skeleton repair	Generated userspace skeleton projects	Installed libbpf header	Detect absent map-link field, remove incompatible assignments, rebuild	Local generated-userspace build repair only
Scheduler extension	Generated `sched_ext` object and five-callback C/eBPF control	Local kernel/toolchain	`bpftool prog loadall`, pinned programs, register/workload/unregister state	Toy FIFO workload only, not scheduler policy quality

pends on whether the implementation actually encodes domain concepts rather than only accepting example syntax.

Static rejection corpus. The static-check harness compiles 28 targeted programs and verifies that every observed outcome matches the expected result. The corpus includes one positive control and 27 expected compiler rejections: 5 lifecycle API misuses, 5 program-signature violations, 3 map type or symbol errors, 4 general type-system errors, 2 symbol-validation errors, 1 config-boundary violation, 1 helper-scope violation, 2 kernel-context allocation violations, 2 perf-event group-bound violations, 1 ring-buffer API misuse, and 1 stack-limit violation. These cases are not a soundness proof, and we are deliberately careful about what they do and do not show. The corpus is author-constructed: each invalid program was written to exercise one specific check and confirmed to be rejected. It is therefore a *diagnostic-contract regression test*. It demonstrates that the checks of Section 5 are implemented, attached to the right error classes, and stable, so that a future compiler change which silently accepts one of these invalid programs, or changes its diagnostic category, will fail the paper-number generation path. It does *not* establish that these checks catch the mistakes developers actually make, nor that they reject errors the C/libbpf path would only surface late at verifier-load or attach time. That stronger claim requires an organic mistake corpus rather than a synthetic one; we make this gap explicit in Section 10 and treat the organic study as the key remaining early-checking experiment.

8.2 Build, load, attach, and compatibility

Example buildability. Table 2 summarizes the example build results. Of 44 examples, 43 compile from KernelScript source.

Table 2: Repository examples at commit 3b19cd2: 41/44 build fully, with one intended safety rejection and two local struct_ops/libbpf skeleton mismatches.

Outcome	Count	Fraction
Examples total	44	100%
KernelScript compile success	43	97.7%
Generated project build success	41	93.2%
Safety rejection	1	2.3%
Struct_ops/libbpf mismatch	2	4.5%

41 then build into generated C/eBPF artifacts. The single compile-time rejection is `safety_demo.ks`, where safety analysis detects 608 bytes of stack usage and halts before C generation. This is a positive result for the early-checking hypothesis: the compiler catches a verifier-relevant problem early and provides a specific repair suggestion.

The two generated build failures are `sched_ext_simple.ks` and `struct_ops_simple.ks`. Both compile to eBPF objects and generate skeletons, but the generated userspace skeleton code references a `struct bpf_map_skeleton.link` field not available in the installed libbpf 1.3.0 headers. The fast-moving struct_ops path therefore exposes a compatibility issue, not a parser or type-checking failure, and it shows that a unified-source language still inherits version skew across bpftool, libbpf, and kernel headers.

Struct_ops object compatibility. To separate that skeleton-header failure from object compatibility, `run_struct_ops_compat.py` compiles

struct_ops_simple.ks with make ebpf-only, compiles a hand-written C/eBPF object with the same minimal tcp-congestion function set, and loads both with one libbpf runner. The runner finds the struct_ops map, calls bpf_map__attach_struct_ops, destroys the returned link, and records detach success. On this host, bpftool is v7.7.0 but libbpf-dev is 1.3.0, and the local struct bpf_map_skeleton header reports skeleton map-link support as no. Without generated skeletons, both the generated object and the C/eBPF object load, attach, and detach in 3/3 and 3/3 trials, respectively. This check does not validate scheduler-extension struct_ops, full struct_ops signature modeling, or workload performance, but it narrows the struct_ops_simple.ks generated userspace build failure to a skeleton compatibility problem for this host.

Struct_ops skeleton repair. The local struct bpf_map_skeleton header reports skeleton map-link support as no. When that field is absent, run_struct_ops_skeleton_repair.py removes the generated skeleton map-link assignments that point at struct_ops links and rebuilds the two affected generated userspace projects. On this host, the original generated userspace builds succeed in 0/2 cases. After the repair, 2/2 build. The script removes 2 assignments across the two examples. The result is a local generated-userspace build repair for the observed map-link mismatch. It does not run the repaired binaries, integrate the decision into the compiler, or prove portability across libbpf versions.

Struct_ops TCP workload. To move beyond load/attach/detach, run_struct_ops_workload.py uses one runner for the generated object and the C/eBPF object. After attaching each tcp-congestion struct_ops map, the runner creates a loopback TCP client/server pair, sets the client socket's TCP_CONGESTION option to the newly registered BPF algorithm, verifies that the socket reports the requested algorithm, transfers 1MiB, and detaches the link. Across 10 trials, the generated object selects the BPF algorithm, transfers the full byte count, and detaches in 10/10 trials. The C/eBPF object does the same in 10/10 trials. Median end-to-end invocation times are 33.3ms and 36.5ms, respectively. The workload is a local socket-level oracle, not a claim about production TCP performance.

Struct_ops callback workload. To check that the TCP workload enters BPF callbacks rather than only selecting a registered algorithm, run_struct_ops_callback_workload.py instruments the generated and C/eBPF tcp-congestion objects with a five-slot BPF array map. Each callback sets its own flag, and the shared runner clears the map before transfer and reads it before detach. The runner also prints a clean-profile callback_clean_required field, while the Python harness computes the profile-specific oracle from the required flag list. Across 10 trials of 4MiB, the generated object completes the byte-transfer oracle in 10/10 trials and sets both cong_avoid and cwnd_event in 10/10 trials. The C/eBPF object matches those results in 10/10 byte-transfer trials and 10/10 callback-oracle trials. The generated object sets cong_avoid and cwnd_event in 10 and 10 trials, respectively, matching 10 and 10 C/eBPF trials. The same script then adds 5% loss to the loopback device with tc netem as a callback-trigger profile and repeats 5 4MiB trials per variant. Under loss, the generated object completes 5/5 byte-transfer trials and sets ssthresh, cong_avoid, set_state, and cwnd_event in 5/5 trials. The C/eBPF object matches with 5/5 byte-transfer trials and 5/5 callback-oracle trials. The undo_cwnd flag is not part of the loss oracle, and it appears in 1 generated loss trials and 1 C/eBPF loss trials. The result is callback-reachability evidence for two local loopback profiles, not coverage of every tcp-congestion callback or a TCP performance result.

Verifier-load matrix. Build success is still weaker than kernel acceptance, so run_verifier_matrix.py loads generated objects with bpftool prog loadall without attaching them. The script counts a load as successful only when bpftool returns success and at least one BPF program is pinned, avoiding false positives from empty program sections. Of 43 objects present after evaluation, 39 load successfully under this stricter criterion. Among the 41 objects from fully built generated projects, 37 load successfully (90.2%) and 4 fail: 1 remaining reference-ownership failure, 1 map creation failure, 1 missing kfunc symbol in kernel BTF, and 1 object for which no program is pinned. Across all generated objects, including those from generated projects that did not fully build, the matrix records no struct_ops argument-type rejection in this run. The current run has 0 other verifier rejections. This result narrows the compatibility claim from "builds" to a measured split between verifier-clean objects and generated objects that still need backend fixes.

Scheduler-extension attach workload. Scheduler extensions are more dangerous to test than packet filters because a successful registration changes the host scheduler. We therefore split this check into a load-only diagnostic and an opt-in attach workload. run_sched_ext_verifier.py compiles a five-callback C/eBPF control scheduler and the generated scheduler-extension object, then runs bpftool prog loadall without scheduler attachment. Both objects verifier-load on this host: the generated object pins 12 programs and the C/eBPF control pins 5 programs, while sched_ext remains disabled.

run_sched_ext_attach.py is intentionally excluded from the default pipeline unless the caller passes an explicit host-scheduler opt-in. The script requires sched_ext to start disabled, registers each scheduler with bpftool struct_ops register, runs a bounded 0.75s CPU workload for 5 trials with up to 4 workers per trial, records each worker's iteration count, requires every worker to make progress, requires sched_ext to remain enabled after registration and every workload trial, unregisters the struct_ops map, and checks that sched_ext returns to disabled. In the checked-in run, both the 5-callback C/eBPF control and the 12-callback generated object pass this oracle: 2 of 2 variants register, keep sched_ext enabled during all trials, unregister cleanly, and

Table 3: Example marker coverage in repository examples shows broad local feature markers but XDP-heavy coverage. A = attempted, KS = KernelScript compiler success, B = generated project build success.

Feature	A	KS	B	Feature	A	KS	B
XDP	38	37	37	TC	5	5	5
Probe	4	4	4	Tracepoint	2	2	2
Perf	2	2	2	Maps	17	16	16
Ringbuf	3	3	3	Dynptr	1	1	1
kfunc	4	4	3	Struct_ops	2	2	0
Tail-call	2	2	2	Userspace	39	39	37

leave 0 rejected sched_ext tasks for the generated variant. The generated scheduler reports a median 37139820 worker-loop iterations versus 31636648 for the C/eBPF control (1.17x), with median per-trial worker-count coefficient of variation 0.008 versus 0.005. The harness provides attach, progress, and fairness sanity evidence for a toy FIFO scheduler, not a scheduling-policy performance result.

Isolated XDP attach matrix. Verifier acceptance is still weaker than attachability. `run_attach_matrix.py` therefore takes the verifier-clean, single-section XDP subset, creates a fresh network namespace and veth pair per object, attaches each object to the namespace-side veth with iproute2, confirms `prog/xdp` in `ip -d link show`, detaches it, and removes the namespace. The matrix attaches and detaches 27 of 27 eligible objects (100.0%), with 0 failures. This broadens the deployment evidence beyond one smoke-test loader, but it is not a packet-behavior or throughput test.

8.3 Generated structure and source footprint

Example marker coverage. The examples exercise many eBPF domains. Table 3 reports attempted examples, examples that pass the KernelScript compiler, and examples that fully build against the local toolchain. These counts should not be read as a complete language coverage proof. They show that the artifact is broader than a tracing-only DSL while making the struct_ops compatibility gap explicit.

Tail-call automation case study. Tail calls are easy to misread in a source-only feature table because the point of the language feature is that KernelScript source does not need to mention `BPF_MAP_TYPE_PROG_ARRAY`, `bpf_tail_call()`, or userspace `bpf_map_update_elem()` registration code. We therefore add a matched case study for 2 local XDP dispatch workloads: a simple `return drop_handler(ctx)` example and a `match`-based protocol classifier. The KernelScript sources total 84 lines, while the matched hand-written C/libbpf baselines total 211 lines (2.51x as many), split across 86 eBPF lines and 125 userspace lines. Compiling the KernelScript sources emits 2 prog-array maps (up to 2 entries), 3 `bpf_tail_call()` sites, 3 explicit fallback returns, and 3 userspace prog-array registration updates. This is a narrow source-footprint and generated-artifact result rather than a throughput claim, but it directly measures the boilerplate that KernelScript removes from source.

The coverage table also highlights a weakness in the current benchmark suite: XDP dominates. That skew is common in early eBPF tools because XDP examples are easy to compile and do not require tracepoint discovery or workload generation. A later performance study should rebalance the corpus so that TC, perf events, ring buffers, kfuncs, and struct_ops each have multiple matched programs with comparable hand-written baselines.

Generated structure. For successful examples, the median KernelScript source size is 31 nonblank noncomment lines. The median generated project size, counting userspace C, eBPF C, optional module C, and Makefile lines but excluding generated `vmlinux.h` and skeleton headers, is 472 lines. The median expansion factor is 11.3x. These measurements show that the compiler emits a median 472 generated source and build lines from 31 KernelScript source lines, giving generated-structure evidence rather than a direct developer-effort result.

The metric is intentionally conservative. It does not say that every generated line is exactly a hand-written line saved, and it does not compare against a ported external application corpus. Within these bounds it shows that KernelScript *centralizes recurring eBPF project structure for this self-authored corpus*: the large ratios are evidence of concern centralization on examples written by the implementation authors, not a corpus-independent claim that KernelScript source is always smaller than hand-written code. The external port portfolio below is the cleanest available check on that boundary, and it shows the ratio does not transfer unchanged to independent code (the KernelScript ports and the external eBPF program bodies have the same line count). Examples such as perf events have particularly high expansion factors because attaching and reading counters requires substantial userspace coordination code relative to the kernel program itself.

Matched source-footprint proxy. The matched source-footprint script covers 11 local workloads that already have hand-written C/eBPF baselines. Counting unique source files, those workloads use 203 maintained KernelScript application lines, while the hand-written C/eBPF objects alone use 254 lines (1.25x as many). Including the C/libbpf runner or loader files used by the matched nontrivial workloads raises the maintained C/libbpf source footprint to 1105 lines (5.44x as many as the KernelScript source). Across per-workload rows, KernelScript is smaller in 8 of 11 rows, while the three rows where it is not smaller are tiny pass or scheduler-control cases with little userspace coordination. The result is a matched source-footprint proxy for the local workload corpus, not a claim about programmer time or external applications.

External source-corpus context. The external source-only scan covers 4 public repositories at pinned commits, 171 selected C/header files, and 38688 nonblank noncomment source lines. The selected corpus contains 82 kernel-side files, 35 userspace files, and 54 local headers. The classifier observes 15 of the tracked feature families: maps appear in 71 files,

XDP in 46, TC in 4, tracepoint in 12, kprobe in 5, perf-event in 5, ring-buffer in 13, struct_ops in 20, scheduler-extension markers in 76, kfunc-related markers in 45, and tail-call markers in 5 files. These rows provide external feature-context evidence for the paper's chosen local program classes. They also narrow the claim: the source scan now includes some explicit tail-call context, but the separate external port portfolio below is still limited to three very small `xdp-tutorial` XDP programs that cover pass, program-selection, and map-counter behavior rather than tail-call dispatch. The honest reading is therefore that external tail-call context is present only in the source scan, not in the port/build/runtime check; the matched tail-call case study above remains the relevant direct check for KernelScript's automatic tail-call support. No external program is translated, built, verifier-loaded, attached, or run by this experiment. A manual spot-check of the lexical classifier matches 8 of 8 sampled files, with 0 false-positive feature labels and 0 false-negative feature labels.

External port/build/runtime check. The external port harness uses three pinned `xdp-tutorial` XDP programs as concrete checks beyond source-only scanning: `basic01-xdp-pass`, `basic02-prog-by-name`, and `basic03-map-counter`. The KernelScript ports total 45 nonblank noncomment lines, while the original external C/eBPF program bodies total 45 lines. This parity is the most important number in the external check, and it directly qualifies the generated-structure result: on code the KernelScript authors did not write, the large source-footprint ratios of the self-authored corpus do not reproduce. The comparison is not perfectly matched—the KernelScript count includes the userspace `main` that loads and attaches the program, whereas the external C count is the eBPF program body alone and relies on `xdp-tutorial`'s shared userspace loader that this count excludes—so the honest reading is that the external port neither confirms a corpus-independent line-count advantage nor shows KernelScript to be larger; it shows that the centralization benefit is corpus-dependent and must be measured per workload rather than assumed. It is also intentionally too small and simple to say anything useful about more orchestration-heavy features such as automatic tail-call dispatch: these three ports are all single-entry XDP examples without prog-array registration or multi-stage dispatch logic. The KernelScript ports build through generated Makefiles, and the original external C/eBPF sources compile directly to BPF objects with clang. All 6 objects attach to isolated veth devices and pass iperf3 traffic in 5 one-second trials. The map-counter workload also increments the XDP_PASS counter key in both variants, with median map update rates of 1.59 and 1.56 million updates/s for KernelScript and C/eBPF, respectively. Receiver-throughput medians range from 18.1–18.3 Gb/s for the KernelScript ports and 17.1–18.0 Gb/s for the original C/eBPF objects. These throughput and counter-rate numbers are descriptive local samples, not a ranking. The check is useful external port sanity evidence, but it remains a small manual XDP portfolio.

Table 4: XDP microbenchmarks and compiler-patch lowering ablation. The experimental compiler patch removes the count gap in this harness.

Bench.	Impl.	Instr.	ns (range)
pass	KernelScript	2	5 (5–6)
pass	C/eBPF	2	6 (5–6)
count	KernelScript	21	12 (12–13)
count	KernelScript compiler patch	11	9 (8–9)
count	C/eBPF	11	9 (9–10)

8.4 Runtime behavior and mechanism isolation

Runtime microbenchmarks and lowering ablation. Table 4 uses BPF_PROG_TEST_RUN for 100000 repetitions over seven trials and reports median average nanoseconds with min–max ranges. The pass rows come from `run_microbench.py`, and the count rows come from `run_compiler_patch_ablation.py`. We intentionally use this single ablation run for all count rows so the current, patched, and C objects share one timing run. The count harness resets the pinned `counts` map before each trial and requires key 0 to equal 100000 afterward. The minimal pass program has the same median in this harness: 2 instructions and 5 (5–6) for KernelScript versus 6 (5–6) for C. The map counter isolates a lowering choice. The current compiler emits lookup plus update, which produces 21 instructions and 12 (12–13). Applying a compiler-source patch that lowers constant array-map increments to a checked lookup plus `__sync_fetch_and_add` reduces the object by 10 instructions (47.6%) and 3ns median (25.0%), matching the hand-written C/eBPF baseline in this harness.

Tail-call microbenchmark. The tail-call performance check uses the same BPF_PROG_TEST_RUN mechanism for 100000 repetitions over 7 trials on two XDP dispatch workloads. It compares generated tail-call objects against hand-written tail-call baselines, then compares tail-call dispatch against a flattened direct-dispatch version of the same logic. In the simple one-target dispatch workload, the generated tail-call object remains close to the hand-written tail-call object at 9 versus 7 instructions and 27 versus 31 ns median (-12.9%), but this pair is small enough that local timing noise obscures any stable tail-versus-flat ordering. In the larger protocol-classifier workload, the generated tail-call object is likewise close to the hand-written tail-call object at 15 versus 9 instructions and 32 versus 31 ns median (3.2%). Comparing the generated tail-call object against the generated flattened variant shows the mechanism cost more directly here: the classifier adds 7 instructions and 5 ns median (18.5%) over the flattened 8-instruction, 27 ns variant. The point of this check is narrow: automatic tail-call lowering stays close to hand-written tail-call code, while the mechanism itself has a visible but small per-invocation cost in the larger local dispatch microbenchmark.

Traffic-driven XDP baseline. To check that the microbenchmark result does not hide an attach-path collapse, `run_xdp_traffic.py` runs the same pass and count programs over real TCP traffic on a veth pair. Each trial creates

a fresh network namespace, attaches the XDP object to the receiver-side veth, starts an iperf3 server in the namespace, and runs a host-side iperf3 client for 1s. The count variants additionally read the `counts` map after each trial and require a positive count value. Over 10 trials, the pass medians are 17.4 (16.0–18.8) Gb/s for KernelScript and 17.8 (14.8–19.2) Gb/s for hand-written C/eBPF. The count medians are 17.2 (16.2–17.9) Gb/s for KernelScript and 17.3 (16.3–18.8) Gb/s for C/eBPF. In this local run, the count medians differ by 0.6% with overlapping ranges. The corresponding XDP count-map invocation rates are 1.50 and 1.50 Mpps. These numbers are local veth/TCP measurements, not NIC-rate claims, and the pass ranges include local run-to-run noise. The point of the run is narrower: it adds a traffic-driven check to the object-level BPF_PROG_TEST_RUN results.

Traffic-driven TC baseline. To add a non-XDP workload, the same traffic harness runs TC ingress pass and count programs attached with `tc qdisc clsact` and a direct-action BPF filter. Over 10 trials, the TC pass medians are 89.9 (83.7–97.5) Gb/s for KernelScript and 92.0 (83.7–103.6) Gb/s for C/eBPF. The TC count medians are 93.0 (83.4–96.8) Gb/s for KernelScript and 90.7 (86.1–95.8) Gb/s for C/eBPF. In this local run, the count medians differ by -2.5% with overlapping ranges. The count-map execution oracle reports TC BPF-invocation rates of 0.27 and 0.26 Mpps, respectively, and all TC trials have zero retransmits in the checked-in run. The result is still local-host evidence, but it directly addresses the XDP-only limitation of the earlier runtime checks.

Longer traffic stress. To check whether the one-second traffic trials hide immediate failures, `run_traffic_stress.py` reruns the same XDP and TC pass/count objects for 3 trials of 5s each. The stress harness reports iperf3 success for all pass/count runs and positive map-count oracles for all count variants, but the XDP stress rows include retransmits and one low generated pass sample. We therefore use this run as short-run sanity evidence, not as a stability claim or throughput ranking. In the count benchmarks, XDP reports 17.3 (16.2–17.6) Gb/s for KernelScript and 15.3 (10.7–17.3) Gb/s for C/eBPF, while TC reports 89.5 (86.0–92.4) Gb/s and 86.1 (85.5–93.7) Gb/s. In this local run, the XDP and TC count medians differ by -13.0% and -3.9%, respectively, with overlapping ranges. The count-map invocation rates remain positive at 1.50 and 1.33 Mpps for XDP, and 0.26 and 0.25 Mpps for TC. The rerun remains a short local stress check with local-host noise.

Perf-event loader and counter workloads. To check generated userspace coordination beyond XDP attachment in one generated-loader path, the perf-event loader script compiles `perf_page_fault.ks`, compiles a matched hand-written C/libbpf loader, and runs each binary with `sudo -n`. Over 20 trials, both loaders attach two perf-event programs, read page-fault and branch-miss counters, and detach cleanly. The generated loader's median end-to-end invocation latency is 15.7 (13.0–24.0)ms, with p90 20.6ms, while the C/libbpf loader reports 18.2 (16.2–27.1)ms and p90 21.1ms. The corresponding median lifecycle invocation rates are 63.9 and 54.9 invocations/s. Because this timing includes process startup, sudo, load, attach, counter read, detach, and teardown, we treat it as generated-loader lifecycle sanity evidence rather than a dispatch-loop throughput claim.

To add a sustained non-XDP object workload, `run_perf_event_counter.py` compiles matched page-fault counter objects and runs both with one libbpf runner. Each trial faults 65536 pages for 4 rounds, then requires the BPF map count to equal the perf counter read. Over 10 trials, both variants report a median 262147 counted events. The generated object reaches 1.02 (0.99–1.04) million events/s, while the C/eBPF object reaches 1.05 (1.03–1.08) million events/s. In this local run, the medians differ by 3.2% with overlapping ranges. These results are lifecycle and page-fault counter evidence, not a broad perf-event throughput study.

Ring-buffer event workload. To exercise an event-delivery path that uses ref-counted ring-buffer records, `run_ringbuf_workload.py` compiles matched XDP objects that reserve a ring-buffer record, write a marker, submit the record, and increment submitted/drop counters. One libbpf runner executes each object through BPF_PROG_TEST_RUN, consumes the ring buffer, and requires submitted events to equal received events with zero drops, bad markers, bad return values, or runner errors. Over 10 trials of 50000 events, both variants report 50000 submitted and 50000 received events with 0 drops. The generated object reaches 2.09 (2.02–2.12) million events/s, while the C/eBPF object reaches 2.18 (2.06–2.21) million events/s. In this local run, the medians differ by 3.9% with overlapping ranges. The result is object-level ring-buffer evidence, and it does not measure generated userspace dispatch-loop throughput.

Generated code size. Among the 41 successful repository examples with disassembly, the median generated eBPF instruction count is 14.5, the midpoint of the two middle objects. Small examples compile to tiny programs, while map-heavy examples generate much larger objects. The metric helps choose where runtime work is needed: examples with many generated map operations are the most likely to diverge from hand-written C.

Execution smoke test. The separate smoke test compiles `smoke_lo.ks`, builds the generated project, and runs the generated loader with sudo. The loader successfully attaches an XDP pass program to the loopback interface and detaches it: `XDP attached to interface: lo` followed by `XDP detached from interface index: 1`. This test is not a benchmark, but it validates that this minimal generated XDP loader can complete a real load/attach/detach lifecycle on `lo` on the test host, complementing the object-level attach matrix.

9 Discussion

The evaluation supports KernelScript's central research direction, but it also clarifies what remains unproven. The strongest

supported claim is a generated-structure claim: one source model can generate the multi-file project structure needed by several eBPF example classes. The test suite, expanded static corpus, and safety demo support a narrower checking claim: the current compiler rejects selected lifecycle, program-signature, map-type, ring-buffer-API, config-boundary, helper-scope, kernel-context allocation, perf-event group-bound, symbol-validation, type-system, and stack-limit errors before kernel loading. The runtime evidence is deliberately smaller. It shows that a minimal pass program has the same instruction count and a comparable BPF_PROG_TEST_RUN median in this local harness, and that the map-update gap is caused by a specific lowering choice, not by the unified source model alone. The compiler-source patch result strengthens that mechanism claim. The XDP attach matrix plus XDP, TC, perf-event, ring-buffer, direct struct_ops, and callback-instrumented struct_ops checks strengthen the local deployment, generated-loader lifecycle, object-workload, object-compatibility, and callback-reachability evidence across five program classes, and the longer traffic rerun adds a small stress check for the XDP and TC cases. The scheduler-extension attach harness adds a bounded host-scheduler progress and fairness proxy for one toy FIFO policy. The artifact still does not establish broad runtime equivalence with hand-written C/libbpf.

The struct_ops failures are instructive. A DSL can hide coordination code, but it cannot hide all ABI and API drift. When bpftool-generated skeletons and installed libbpf headers disagree, the generated userspace C fails. A direct libbpf runner can load, attach, and detach the generated tcp-congestion struct_ops object on this host, and a loopback TCP socket can select the registered BPF congestion-control algorithm and transfer data. A callback-flag variant shows that this workload reaches `cong_avoid` and `cwnd_event` in both generated and C/eBPF objects, and the loss-injected profile also reaches `ssthresh` and `set_state`. Thus, the measured failure is not simply an invalid object, unusable TCP algorithm, or entirely unreached callback path. The skeleton-repair experiment shows that one local map-link mismatch can be handled with a version-aware generated-header repair, but a production compiler would still need to integrate that decision into generation, dependency checks, or containerized toolchains. The scheduler-extension experiments add a sharper boundary: a `BPF_PROG`-wrapped C/eBPF sched_ext object and the generated sched_ext object both verifier-load, and an opt-in harness can register both toy schedulers, run a bounded CPU workload, and unregister them cleanly on this host. This removes the previous load-only gap for the toy scheduler, but it is still not a scheduler-performance or production-policy study. From a systems-research perspective, this is part of the contribution: the artifact exposes where a unified model centralizes structure and where the underlying Linux BPF stack still leaks through.

The expansion-factor and source-footprint metrics should also be interpreted carefully. Generated C is not automatically better than hand-written C. It can be verbose, unidiomatic, or harder to debug, so the practical benefit depends on diagnostic quality and source-level mapping from generated errors back to KernelScript source. The matched source-footprint proxy improves on a generated-only expansion factor, but it still uses local baselines and line counts rather than a controlled developer study. Future work should therefore measure time to fix injected errors, quality of compiler messages, and whether developers can understand generated verifier failures. The external corpus scan should be interpreted just as carefully: it measures feature overlap in pinned public source trees, not whether KernelScript can compile those programs or preserve their behavior. The external port experiment improves that gap for a small pinned XDP portfolio by adding compile/build/attach/runtime evidence, but it is still a manual same-repository port set and cannot justify a broad external-application portability claim.

10 Threats to Validity

No developer-time study. The source-footprint proxy compares KernelScript source against the hand-written C/eBPF baselines and C/libbpf runners used by the local matched workload corpus. This proxy is more direct than comparing KernelScript input with generated output, but it still measures lines rather than task time, debugging effort, review burden, or expert familiarity. An expert could write smaller C for some examples, especially tiny XDP pass/drop programs. Conversely, generated code omits comments and may be denser than maintainable production C. The right interpretation is therefore "matched source footprint centralized for these local workloads," not "developer time saved."

Repository examples may be biased. The corpus comes from the KernelScript artifact repository, so it naturally reflects the features the implementation authors wanted to demonstrate. The corpus is useful for an artifact study because it is reproducible and covers many features, but it is not a representative sample of all eBPF applications. A stronger corpus would include programs from Cilium, bcc/libbpf-tools, production XDP filters, scheduler extensions, and observability agents, then reimplement each in KernelScript and C/libbpf. The external source scan adds independent feature context, but it is source-only and does not remove this threat. The external port portfolio lowers the risk for three small XDP tutorial workloads but leaves the broader corpus threat in place. This bias bears directly on the generated-structure and source-footprint results: the large expansion and footprint ratios are measured on author-written examples, and on the one independent workload set the KernelScript and external eBPF line counts are equal. We therefore scope the centralization claim to "concern centralization for this corpus" and do not claim a corpus-independent reduction in source size; establishing the latter requires the broader reimplement-in-both study described above.

Static checks are targeted and synthetic, not organic or complete. The static rejection corpus is an author-constructed regression test for selected compiler checks, not a formal proof of lifecycle soundness or memory safety, and not a sample of

real developer mistakes. Because each program was written to be rejected, the corpus shows that the checks of Section 5 are wired and stable, but it cannot by itself show that early checking pays off in practice: it does not cover all invalid attach orders, helper contracts, context-dependent verifier failures, or semantic equivalence with hand-written C/libbpf, and it cannot quantify how often KernelScript moves a real failure earlier than the C/libbpf path. The experiment that would settle this is an *organic* early-checking study, which we have not yet run: collect real eBPF mistakes (for example from the revision history and issue trackers of `xdp-tutorial`, `libbpf-bootstrap`, and Cilium, or via mutation of working programs), port each to KernelScript, and report the failure stage in each toolchain—KernelScript compile-time rejection versus C/libbpf clang-compile, verifier-load, or attach-time failure. Such a study would convert the current regression test into a measured claim about which late failures KernelScript actually shifts earlier; until it is done, the early-checking contribution should be read as "these checks exist and are stable," not "these checks catch the bugs developers hit."

The compiler patch is narrow and experimental. The patch used in the ablation is a tracked compiler-source patch applied to a copied compiler tree, not an upstream change in the evaluated KernelScript commit. The patch is intentionally limited to constant increments on integer array maps, where lookup followed by in-place atomic add preserves the count benchmark's behavior. Broader claims would require upstreaming or otherwise integrating the rule and rerunning verifier, instruction-count, and runtime checks across hash, per-CPU, and structured map values.

Local traffic is not broad workload validation. The verifier matrix loads generated eBPF objects and therefore catches failures beyond C compilation. The attach matrix goes further for verifier-clean single-section XDP objects by installing and removing them on isolated veth devices, and the traffic benchmarks send TCP through matched XDP and TC pass/-count programs. The generated perf-event loader test adds one repository-example loader that opens perf events, attaches BPF programs, reads counters, detaches cleanly, and records end-to-end invocation latency against a C/libbpf loader. The perf-event counter and ring-buffer benchmarks add sustained object workloads, but they use a shared libbpf runner and therefore do not measure generated-dispatch-loop throughput. The struct_ops compatibility checks also use shared libbpf runners. The TCP workload validates socket-level algorithm selection and byte transfer through the generated and C/eBPF tcp-congestion objects, and the callback-flag workload validates that a clean loopback transfer reaches `cong_avoid` and `cwnd_event` in both variants. Its loss-injected profile also reaches `ssthresh`, `cong_avoid`, `set_state`, and `cwnd_event`. The skeleton repair validates a local generated-userspace build fix for the observed map-link mismatch. The scheduler-extension attach harness registers the generated and C/eBPF toy FIFO schedulers and runs a short CPU workload under sched_ext before unregistering them. These checks do not measure production TCP performance, cover every tcp-congestion-control callback, evaluate scheduler-policy quality, prove compatibility across libbpf versions, or measure generated-loader behavior. However, the traffic runs, including the 5s stress rerun, are local-host veth evidence, not NIC-rate performance or long-duration soak tests. A complete deployment study would run each example in a VM with known kernel configuration, representative workloads, and explicit cleanup checks after detach.

Kernel version skew matters. The struct_ops failures show that results depend on the alignment between kernel headers, bpftool, generated skeleton code, and libbpf headers. The repair experiment covers the local map-link mismatch, but it is not a substitute for broader version testing. The threat is not incidental: eBPF programming is tightly coupled to kernel versions. A production KernelScript compiler should record the target kernel ABI, check libbpf feature support before generation, and fail early with a dependency diagnostic when a generated feature cannot be compiled locally.

11 Related Work

eBPF and XDP foundations. XDP established the in-kernel programmable packet-processing model that KernelScript targets, executing verified eBPF at the earliest receive point [7], and the broader eBPF/XDP design space and its application classes have since been surveyed in detail [12, 11]. C/libbpf remains the reference workflow: it exposes the full API surface and integrates BTF and Compile Once, Run Everywhere (CO-RE), but it requires developers to manage kernel code, userspace loaders, skeletons, maps, and build logic as separate artifacts [9]. KernelScript keeps this toolchain underneath but moves the multi-artifact structure into one type-checked source.

Verification and optimization of kernel extensions. A separate line of work hardens or optimizes the eBPF programs themselves. PREVAIL replaces parts of the in-kernel verifier with an abstract-interpretation analysis that is more precise on larger programs [6], and K2 uses program synthesis to produce smaller, safe eBPF objects from existing ones [13]. hXDP compiles eBPF for execution on FPGA NICs [4]. These systems operate on the eBPF *bytecode* after it exists; KernelScript is complementary and earlier in the pipeline, generating the bytecode (and the surrounding loader and build) and moving selected checks ahead of the verifier rather than replacing or re-optimizing it. Our compiler-source ablation (Section 7) touches the same instruction-lowering concerns that K2 optimizes, but as a single hand-identified rule, not an optimizer.

Languages and bindings for eBPF. bpftrace offers a concise high-level language but deliberately narrows the target to tracing, excluding XDP, TC, struct_ops, and kfunc workflows [3]. Host-language libraries such as Aya (Rust) and cilium/ebpf (Go) improve userspace integration and type safety within their host language, but preserve the split between the eBPF program and the controlling application [1, 5]. KernelScript differs on both axes: it is compiled and multi-domain

rather than tracing-only, and it unifies the program and its controller in one source rather than binding a separately written program from a host language.

Domain-specific languages for packet and dataplane processing. The idea of compiling one typed description into a split low-level implementation has strong precedent outside eBPF. P4 compiles a single protocol-independent program into target-specific parser and match-action pipelines [2], and the Click modular router composes packet processing from typed elements [8]. KernelScript attacks an analogous but distinct unification problem: unlike a P4 pipeline, an eBPF application must also generate its userspace coordinator and survive a late, general-purpose kernel verifier and fast-moving libbpf/bpftool ABIs, which is the source of the compatibility limits we measure.

Typestate and resource-lifecycle typing. KernelScript's lifecycle discipline (Section 5) is a lightweight instance of typestate, the idea that an object's legal operations depend on its current state [10]. We enforce only the producer/consumer handle distinction rather than a full flow-sensitive typestate analysis, so this connection is a design influence and a direction for future work rather than a claim of equivalent guarantees.

Positioning. KernelScript therefore occupies a point not covered by the above: a compiled, multi-domain, single-source language whose research question is whether eBPF application structure can be made explicit enough for a compiler to generate the low-level project *while preserving* the existing Linux BPF toolchain. The evaluated implementation is best read as both an implementation and a concrete design artifact for that point in the space.

12 Conclusion

KernelScript is an early but useful prototype for studying typed unified-source eBPF development. The checked-in artifact supports three bounded claims. First, the evaluated compiler can centralize recurring project structure for this repository corpus: 43 of 44 examples compile, 41 build, and the median successful example expands from 31 KernelScript source lines to 472 generated source and build lines. A matched source-footprint proxy adds 11 local workload rows in which unique maintained KernelScript sources total 203 lines versus 1105 hand-written C/libbpf lines when baseline runners are included. A pinned external source-corpus scan covers 171 files across 4 public eBPF repositories and observes 15 tracked feature families, giving source-only context for the local evaluation scope. A small manual external portfolio from `xdp-tutorial` also builds through KernelScript's generated Makefiles and runs alongside directly compiled original external C/eBPF objects over 5 traffic trials. Second, the implementation exposes selected errors before kernel loading: the targeted static corpus includes 27 expected invalid programs, and the evaluation keeps those diagnostics executable. Third, the generated artifacts are not merely syntactic outputs on this host: verifier loading, isolated XDP attachment, matched XDP/TC, perf-event, ring-buffer, and tcp-congestion struct_ops checks exercise local load, attach, event, traffic, and callback paths against explicit oracles, and the scheduler-extension harness exercises one bounded host-scheduler attach/workload path.

The contribution is therefore a reproducible prototype study rather than a finished eBPF replacement. It shows how a compiler-checked unified programming model can make selected cross-boundary eBPF structure explicit while still lowering through libbpf and bpftool. The remaining gates are also clear: version-aware skeleton handling needs upstream integration and broader version coverage, scheduler-extension support now has only a toy bounded workload rather than policy or performance evidence, broader callback-level TCP behavior remains incompletely tested, generated-loader dispatch-loop throughput needs stronger workloads, and the experimental map-update compiler patch needs upstream integration plus broader semantic and runtime validation.

References

[1] Aya contributors. Aya: Rust ebpf library. `https://github.com/aya-rs/aya`, 2026.

[2] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. P4: Programming protocol-independent packet processors. *ACM SIGCOMM Computer Communication Review*, 44(3):87–95, 2014.

[3] bpftrace developers. bpftrace: High-level tracing language for linux ebpf. `https://github.com/bpftrace/bpftrace`, 2026.

[4] Marco Spaziani Brunella, Giacomo Belocchi, Marco Bonola, Salvatore Pontarelli, Giuseppe Siracusano, Giuseppe Bianchi, Aniello Cammarano, Alessandro Palumbo, Luca Petrucci, and Roberto Bifulco. hXDP: Efficient software packet processing on FPGA NICs. In *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 973–990, 2020.

[5] Cilium contributors. cilium/ebpf: ebpf library for go. `https://github.com/cilium/ebpf`, 2026.

[6] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. Simple and precise static analysis of untrusted Linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 1069–1084, 2019.

[7] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David

Ahern, and David Miller. The eXpress Data Path: Fast programmable packet processing in the operating system kernel. In *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, pages 54–66, 2018.

[8] Eddie Kohler, Robert Morris, Benjie Chen, John Jannotti, and M. Frans Kaashoek. The Click modular router. *ACM Transactions on Computer Systems*, 18(3):263–297, 2000.

[9] libbpf developers. libbpf: a bpf co-re userspace library. `https://github.com/libbpf/libbpf`, 2026.

[10] Robert E. Strom and Shaula Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering*, SE-12(1):157–171, 1986.

[11] The Linux Kernel Documentation Project. eBPF documentation. `https://docs.kernel.org/bpf/`, 2026.

[12] Marcos A. M. Vieira, Matheus S. Castanho, Racyus D. G. Pacífico, Elerson R. S. Santos, Eduardo P. M. Câmara Juńior, and Luiz F. M. Vieira. Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications. *ACM Computing Surveys*, 53(1):1–36, 2020.

[13] Qiongwen Xu, Michael D. Wong, Tanvi Wagle, Srinivas Narayana, and Anirudh Sivaraman. Synthesizing safe and efficient kernel extensions for packet processing. In *Proceedings of the 2021 ACM SIGCOMM Conference*, pages 50–64, 2021.